

main.py



Share

Run

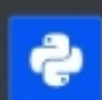
Output

Clear

```
43     return (left << 8) | right
44
45 # -----
46 # Main Program
47 # -----
48
49 key = 0x3A94
50 keys = generate_keys(key)
51
52 plaintext = 0x1234
53
54 print("Plaintext :", hex(plaintext))
55
56 start = time.time()
57 ciphertext = encrypt_block(plaintext, keys)
58 end = time.time()
59
60 print("Ciphertext :", hex(ciphertext))
61 print("Encryption Time :", end - start, "seconds")
62
63 start = time.time()
64 decrypted = decrypt_block(ciphertext, keys)
65 end = time.time()
66
67 print("Decrypted Text :", hex(decrypted))
68 print("Decryption Time :", end - start, "seconds")
```

Plaintext : 0x1234
Ciphertext : 0x7ba8
Encryption Time : 4.76837158203125e-06 seconds
Decrypted Text : 0x1234
Decryption Time : 4.291534423828125e-06 seconds

=== Code Execution Successful ===



main.py



Share

Run

Output

Clear

```
1 import time
2
3 # Simple XOR Encryption
4 def xor_encrypt(data, key):
5     result = ""
6     for i in range(len(data)):
7         result += chr(ord(data[i]) ^ key)
8     return result
9
10 # ECB Mode
11 def ecb_encrypt(text, key):
12     return xor_encrypt(text, key)
13
14 def ecb_decrypt(cipher, key):
15     return xor_encrypt(cipher, key)
16
17 # CBC Mode
18 def cbc_encrypt(text, key, iv):
19     cipher = ""
20     prev = iv
21
22     for ch in text:
23         x = ord(ch) ^ prev
24         c = x ^ key
25         cipher += chr(c)
26         prev = c
```

```
Plaintext : NETWORK SECURITY
ECB Cipher : b'AJ[X@]D/\JLZ]F[V'
ECB Decrypted : NETWORK SECURITY
ECB Time : 4.00543212890625e-05
CBC Cipher : b'X\x12I\x11Q\x0cHg;q=g:|'q"
CBC Decrypted : NETWORK SECURITY
CBC Time : 2.1219253540039062e-05
CFB Cipher : b'X\x12I\x11Q\x0cHg;q=g:|'q"
CFB Decrypted : NETWORK SECURITY
CFB Time : 2.3126602172851562e-05
OFB Cipher : b'X\BNYK]9E\ULDPB@'
OFB Decrypted : NETWORK SECURITY
OFB Time : 2.2172927856445312e-05
=== Code Execution Successful ===
```



main.py



Share

Run

Output

Clear

```
1 import time
2
3 # Function to find GCD
4 def gcd(a, b):
5     while b != 0:
6         a, b = b, a % b
7     return a
8
9 # Function to find modular inverse
10 def mod_inverse(e, phi):
11     for d in range(1, phi):
12         if (e * d) % phi == 1:
13             return d
14     return None
15
16 # Fast Modular Exponentiation
17 def power(base, exp, mod):
18     result = 1
19     base = base % mod
20
21     while exp > 0:
22         if exp % 2 == 1:
23             result = (result * base) % mod
24
25         exp = exp // 2
26         base = (base * base) % mod
```

```
Public Key : (7, 187)
Private Key : (23, 187)

Original Message : 88
Encrypted Message : 11
Encryption Time : 2.86102294921875e-06
Decrypted Message : 88
Decryption Time : 2.384185791015625e-06

=== Code Execution Successful ===
```




main.py



Share

Run

Output

Clear

```
1 import time
2
3 # Fast Modular Exponentiation
4 def power(base, exp, mod):
5     result = 1
6     base = base % mod
7
8     while exp > 0:
9         if exp % 2 == 1:
10             result = (result * base) % mod
11
12         exp = exp // 2
13         base = (base * base) % mod
14
15     return result
16
17 # Public values
18 prime = 23
19 generator = 5
20
21 # Private keys
22 alice_private = 6
23 bob_private = 15
24
25 # Public keys
26 alice_public = power(generator, alice_private, prime)
```

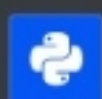
```
Prime Number : 23
Generator : 5

Alice Public Key : 8
Bob Public Key : 19

Alice Shared Key : 2
Bob Shared Key : 2

Key Exchange Successful
Execution Time : 3.814697265625e-06 seconds

=== Code Execution Successful ===
```



main.py



 Share

Run

Output

Clear

```

50 start = time.time()
51
52 # Public Keys
53 alice_public = scalar_mult(alice_private, G)
54 bob_public = scalar_mult(bob_private, G)
55
56 # Shared Secret
57 alice_shared = scalar_mult(alice_private, bob_public)
58 bob_shared = scalar_mult(bob_private, alice_public)
59
60 end = time.time()
61
62 print("Base Point :", G)
63
64 print("\nAlice Public Key :", alice_public)
65 print("Bob Public Key :", bob_public)
66
67 print("\nAlice Shared Key :", alice_shared)
68 print("Bob Shared Key :", bob_shared)
69
70 if alice_shared == bob_shared:
71     print("\nKey Exchange Successful")
72 else:
73     print("\nKey Exchange Failed")
74
75 print("Execution Time :", end - start, "seconds")
    
```

```

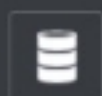
^ Base Point : (5, 1)

Alice Public Key : (10, 6)
Bob Public Key : (0, 6)

Alice Shared Key : (6, 3)
Bob Shared Key : (6, 3)

Key Exchange Successful
Execution Time : 2.9325485229492188e-05 seconds

=== Code Execution Successful ===
    
```



JS

TS

GO

